WebScrapeX

Design Intelligence Platform

Web Scraping & Analysis Application

Project Report for College Submission

Generated: May 4, 2026

Next.js 16 | React 19 | TypeScript | Supabase | Puppeteer | Google AI

Table of Contents

1. Project Overview

2. Technology Stack

3. Project Structure

4. Core Features

5. System Architecture

6. Pages & Functionality

7. API Routes

8. Database Schema

9. Key Components

10. Conclusion

1. Project Overview

> **WebScrapeX** is a modern, production-ready web scraping and design intelligence platform built with Next.js 16, TypeScript, and Supabase. It enables users to extract and analyze design systems from any website, including colors, fonts, images, technologies, and metadata.

Project Objectives

- Extract design elements (colors, fonts, images) from any website
- Detect technologies and frameworks used by websites
- Provide SEO analysis and content insights
- Offer AI-powered natural language data extraction
- Enable API access for developer integration
- Store and organize scraped data in user collections

Key Statistics

Metric	Value
Framework	Next.js 16.1.1 (App Router)
Language	TypeScript 5
Total Pages	11 Pages
Components	40+ Components
API Routes	10+ Endpoints
Dependencies	75+ Packages

2. Technology Stack

Frontend Technologies

Next.js 16

React framework with App Router for SSR, API routes, and file-based routing

React 19

UI library for building interactive components with hooks and context

TypeScript 5

Type-safe JavaScript for better code quality and developer experience

Tailwind CSS 3.4

Utility-first CSS framework for rapid, responsive UI development

Framer Motion 12

Animation library for smooth transitions and micro-interactions

Radix UI

Accessible, unstyled UI components (Dialog, Tabs, Dropdown, etc.)

Backend Technologies

Supabase

PostgreSQL database, authentication, real-time subscriptions, and file storage

Cheerio 1.1

Fast HTML parser for static content extraction (Fast Mode scraping)

Puppeteer 22

Headless Chrome for dynamic content, screenshots, and PDF generation

Google Generative AI

AI integration for natural language data extraction with Gemini

Puppeteer Extra + Stealth

Bypasses bot detection for reliable scraping of protected sites

Mozilla Readability

Content extraction engine for clean article parsing

3. Project Structure

> The project follows Next.js 16 App Router conventions with a clean separation of concerns between pages, components, and core libraries.

Directory Overview

Directory	Purpose
`app/`	Next.js App Router pages and API routes
`components/`	Reusable React components (40+ components)
`lib/`	Core libraries (scraper, database, utilities)
`hooks/`	Custom React hooks (useAuth, useResponsive, etc.)
`scripts/`	Utility scripts (queue worker, cleanup)
`public/`	Static assets and PWA files

App Directory Structure

Path	Description
`app/page.tsx`	Home/Landing page
`app/layout.tsx`	Root layout with providers
`app/api/`	API route handlers
`app/result/[id]/`	Dynamic result pages
`app/docs/`	Documentation page
`app/collections/`	User's scrape history

`app/ai-scraper/`	AI-powered extraction
`app/login/`	Authentication pages

4. Core Features

Hybrid Scraping

Fast mode (Cheerio) + Deep mode (Puppeteer) for comprehensive extraction

Multiple Extractors

Metadata, colors, fonts, technologies, social data, LinkedIn profiles

AI Extraction

Natural language prompts for structured data extraction without selectors

Background Processing

Queue system for non-blocking operations with real-time updates

Intelligent Caching

In-memory caching for repeated requests and faster response times

Rate Limiting

Per-user request limits to prevent abuse (100 req/min)

SEO Analysis

Comprehensive SEO audit with actionable recommendations

Export Options

JSON, PDF, color guides, and asset downloads

Screenshot & PDF

Full-page screenshots and PDF snapshots of scraped pages

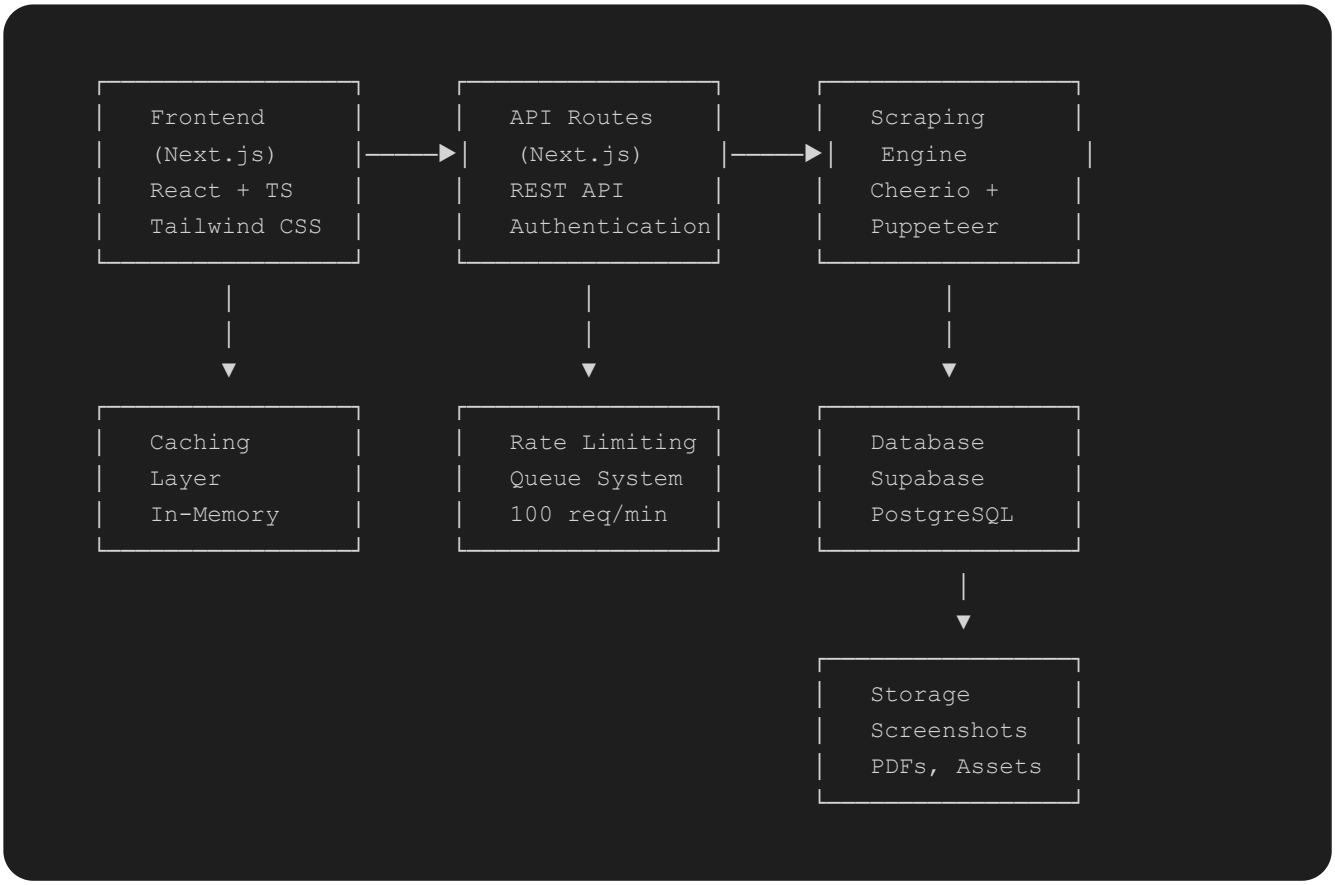
OAuth Authentication

Google and GitHub login with email/password support

Scraping Modes

Mode	Speed	Data Extracted
Full	~30-60s	Everything: colors, fonts, images, tech, metadata, SEO
Social	~5-10s	Open Graph, Twitter Cards, social metadata
Content	~10-15s	Article content, readability analysis, headings
Design	~20-30s	Colors, fonts, images, CSS files

5. System Architecture



Data Flow

1. **User Request:** User submits URL via the web interface
2. **Authentication:** System validates user session and permissions
3. **Rate Check:** Ensures user hasn't exceeded rate limits
4. **Cache Check:** Returns cached result if URL was recently scraped
5. **Queue Job:** Scraping job added to background queue
6. **Scraping:** Engine extracts data using appropriate mode
7. **Processing:** Colors, fonts, technologies extracted and processed
8. **Storage:** Results saved to database and assets to storage
9. **Response:** Frontend polls for results and displays

6. Pages & Functionality

1 Home Page

`Route: /`

Purpose: Main dashboard and landing page for the application.

- Hero section with animated text reveal
- URL input form for initiating scrapes
- Scrape mode selector (Full, Social, Content, Design)
- Feature showcase cards
- User's recent scrape history
- Analytics dashboard for logged-in users

2 Documentation Page

`Route: /docs`

Purpose: Comprehensive user guide and API documentation.

- User Guide: 3-step process (Input → Analyze → Export)
- Standard API: Authentication and key management
- AI Extraction: Natural language prompt documentation

3 Collections Page

`Route: /collections`

Purpose: Archive of all previously scraped websites.

- Grid display of history cards

- Website thumbnails, titles, and dates
- Delete functionality
- Requires authentication

4 API Access Page

`Route: /api-access`

Purpose: Developer API integration and key management.

- API key reveal/regenerate/copy
- Proxy rotation settings
- Webhook configuration
- Code examples (cURL, Node.js, ActivePieces)

5 AI Scraper Page

`Route: /ai-scraper`

Purpose: AI-powered natural language data extraction.

- URL input for target website
- Natural language prompt textarea
- JSON output with syntax highlighting
- No CSS selectors required

6 Result Page

`Route: /result/[id]`

Purpose: Display detailed scrape results with multiple cards.

- MetadataCard: Title, description, keywords
- ColorPaletteCard: Extracted colors
- FontsCard: Detected fonts
- ImageGalleryCard: All images
- TechCard: Technologies detected
- SEOCard: SEO audit results
- ContentAnalysisCard: Article analysis
- SocialCard: Open Graph data
- LinksCard: Internal/external links
- FloatingExportButton: Export options

7 Settings Page

`Route: /settings`

Purpose: User account configuration and preferences.

- General: Theme toggle, export defaults
- Billing: Subscription management
- Account: Profile information
- Notifications: Email preferences

8 Authentication Pages

`Routes: /login, /signup, /forgot-password`

Purpose: User authentication and account management.

- Email/password login and registration
- OAuth providers (Google, GitHub)
- Password recovery
- Session management

7. API Routes

Endpoint	Method	Purpose
`/api/health`	GET	Health check for monitoring
`/api/scrape`	POST	Initiate a new scrape job
`/api/scrape/content`	GET	Retrieve scraped content by ID
`/api/v2/scrape`	POST	Enhanced scraping with mode selection
`/api/v2/scrape/ai`	POST	AI-powered data extraction

API Request Example

```
POST /api/v2/scrape
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json

{
  "url": "https://example.com",
  "mode": "full"
}

Response:
{
  "id": "uuid-here",
  "status": "processing"
}
```

AI Extraction Example

```
POST /api/v2/scrape/ai
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json

{
  "url": "https://store.example.com/products",
```

```
  "prompt": "Extract all product names, prices, and ratings"
}
```

8. Database Schema (Supabase)

websites Table

Column	Type	Description
id	uuid	Primary key
user_id	uuid	Foreign key to auth.users
url	text	Scraped URL
status	text	pending, processing, completed, failed
created_at	timestamp	Creation time
completed_at	timestamp	Completion time

metadata Table

Column	Type	Description
id	uuid	Primary key
website_id	uuid	Foreign key to websites
title	text	Page title
description	text	Meta description
keywords	text[]	Meta keywords array
color_palette	text[]	Extracted colors
fonts	text[]	Extracted fonts
technologies	text[]	Detected tech stack

assets Table

Column	Type	Description
id	uuid	Primary key
website_id	uuid	Foreign key to websites
file_type	text	image, css, js
url	text	Asset URL

9. Key Components

Layout Components

Component	Purpose
Sidebar	Navigation sidebar with all page links
Header	Page header with title, URL, and actions
DynamicBackground	Animated gradient backgrounds

Card Components (13 total)

Component	Displays
MetadataCard	Page title, description, favicon, keywords
ColorPaletteCard	Extracted color palette with hex codes
FontsCard	Detected fonts with preview
ImageGalleryCard	Grid gallery of all images
TechCard	Detected technologies and frameworks
SEOCard	SEO audit with recommendations
SocialCard	Open Graph and Twitter Card data
ContentAnalysisCard	Word count, reading time, headings
LinkedInCard	LinkedIn profile extraction
InsightsCard	Design intelligence scores
LinksCard	Internal and external links
AssetsCard	CSS and JS file listings

RawHTMLCard	Full HTML source viewer

Core Libraries (lib/)

File	Purpose
scraper.ts	Main scraping engine with hybrid mode support
supabase.ts	Database client and admin client
cache.ts	In-memory caching layer
rateLimiter.ts	Request rate limiting logic
queue.ts	Background job queue system
logger.ts	Structured logging utility
retry.ts	Exponential backoff retry logic

WebScrapeX Project Report

College Project Submission

Built with Next.js 16, React 19, TypeScript, and Supabase

© 2026 WebScrapeX Team